

SKILL WEEK 9 - SESSION 17

Task 1 :- To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listing

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

struct Job {
    int id;
    pid_t pid;
    char state[20];
};

int main() {
    struct Job jobs[2];
    int count = 0;
    pid_t pid;

    printf("Active Job Listing\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        sleep(3);
        exit(0);
    }

    jobs[count].id = 1;
    jobs[count].pid = pid;
    snprintf(jobs[count].state,
             sizeof(jobs[count].state),
             "RUNNING");
    count++;

    printf("\nJob ID\tPID\tSTATE\n");

    for (int i = 0; i < count; i++) {
        printf("%d\t%d\t%s\n",
              jobs[i].id,
              jobs[i].pid,
              jobs[i].state);
    }

    printf("\nActive jobs: %d\n", count);
    printf("Job information retrieved successfully.\n");

    waitpid(pid, NULL, 0);

    snprintf(jobs[0].state,
             sizeof(jobs[0].state),
             "COMPLETED");

    printf("\nUpdated Job Status:\n");
    printf("%d\t%d\t%s\n",
          jobs[0].id,
          jobs[0].pid,
          jobs[0].state);

    return 0;
}
```

The output displays the active job ID, process ID, and RUNNING state, followed by the updated COMPLETED state after the job

finishes.

```
root@HP:~/skill_week9# gcc task1_list_jobs.c -o task1_list_jobs
root@HP:~/skill_week9# ./task1_list_jobs
Active Job Listing

Job ID  PID    STATE
1       492    RUNNING

Active jobs: 1
Job information retrieved successfully.

Updated Job Status:
1       492    COMPLETED
root@HP:~/skill_week9# |
```

Task 2 :- To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```

GNU nano 7.2 task2_foreground_switch.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int status;

    printf("Foreground Job Switching Demonstration\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Target job started. PID = %d\n", getpid());

        sleep(2);

        printf("Target job completed.\n");
        exit(0);
    }

    printf("Parent identified target job PID = %d\n", pid);
    printf("Transferring execution control to the target job.\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("Foreground job completed successfully.\n");
        printf("Job state updated to COMPLETED.\n");
    }

    return 0;
}

```

```

1 492 COMPLETED
root@HP:~/skill_week9# nano task2_foreground_switch.c
root@HP:~/skill_week9# gcc task2_foreground_switch.c -o task2_foreground_switch
root@HP:~/skill_week9# ./task2_foreground_switch
Foreground Job Switching Demonstration
Parent identified target job PID = 502
Transferring execution control to the target job.
Target job started. PID = 502
Target job completed.
Foreground job completed successfully.
Job state updated to COMPLETED.
root@HP:~/skill_week9# |

```

SESSION 18

Task 1 :- To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

```
GNU nano 7.2 task3_signals.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig) {
    if (sig == SIGINT)
        printf("\nSIGINT received and handled.\n");

    if (sig == SIGTERM)
        printf("SIGTERM received and handled.\n");
}

int main() {
    sigset_t mask;

    printf("Signal Delivery Demonstration\n");
    printf("Process ID: %d\n", getpid());

    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);

    sigemptyset(&mask);
    sigaddset(&mask, SIGINT);

    sigprocmask(SIG_BLOCK, &mask, NULL);

    printf("SIGINT is temporarily blocked.\n");
    printf("Signal mask configured successfully.\n");

    sleep(2);

    sigprocmask(SIG_UNBLOCK, &mask, NULL);

    printf("SIGINT unblocked.\n");
    printf("Signal handling is active.\n");

    raise(SIGTERM);

    return 0;
}
```

The output displays the process ID, signal mask configuration, SIGINT unblocking, and successful handling of the SIGTERM

```
root@HP:~/skill_week9# nano task3_signals.c
root@HP:~/skill_week9# gcc task3_signals.c -o task3_signals
root@HP:~/skill_week9# ./task3_signals
Signal Delivery Demonstration
Process ID: 511
SIGINT is temporarily blocked.
Signal mask configured successfully.
SIGINT unblocked.
Signal handling is active.
SIGTERM received and handled.
root@HP:~/skill_week9# |
signal.
```

Task 2 :- To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int status;

    printf("Stopped Job Recovery Demonstration\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Job started. PID = %d\n", getpid());
        printf("Job is running in the background.\n");

        sleep(2);

        printf("Job completed after resume.\n");
        exit(0);
    }

    printf("Job stored with PID = %d\n", pid);
    printf("Job state: RUNNING\n");

    sleep(1);
    //-----

    printf("Sending continue signal to the job...\n");
    kill(pid, SIGCONT);

    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("Job state updated: COMPLETED\n");
        printf("Background recovery successful.\n");
    }

    return 0;
}
```

The output displays the job PID, sends the continue signal, and shows the job state changing to COMPLETED after successful

```
root@HP:~/skill_week9# gcc task4_resume_jobs.c
root@HP:~/skill_week9# ./task4_resume_jobs
Stopped Job Recovery Demonstration
Job stored with PID = 519
Job state: RUNNING
Job started. PID = 519
Job is running in the background.
Sending continue signal to the job...
Job completed after resume.
Job state updated: COMPLETED
Background recovery successful.
root@HP:~/skill_week9# |
```

recovery.

Observation :- Signal handling and job recovery were successfully tested, demonstrating signal control, process status management, and background process recovery.